

LangGraph AI 에이전트 개발_1일차

한 기 영

(주)데이터인사이트

bit.ly/li_ai_agent

과정 개요

- ✓ LLM 이해
- ✓ AI Agent 기초
- ✓ AI Agent 기본 패턴
- ✓ 도구와 메모리

LLM 이해

| LLM 특징

다음 문장을 완성해 봅시다.

“인공지능은 생각보다 ____.”

LLM의 중요한 특징

생성형 AI: 확률기반 답변 생성



LLM의 중요한 특징

✓ “나는 아침에 ____”

- 다음 단어를 예측(생성) 하기 위한 **확률 분포**

후보 단어	확률
밥을	0.4
학교에	0.25
운동을	0.15
일찍	0.1
고양이를	0.05
빨래를	0.02
잔다	0.02
청소를	0.01

확률 분포 제어하기 : 프롬프트

✓ 구체적인 질문 → 구체적인 답변

- 생성 모델 : **확률기반 답변** 생성
- 구체적인 질문 → 답변의 확률분포 → 답변 범위

오늘 점심 메뉴 추천해줘.

Gen AI

우리가 예상하는 답변 영역

답변1
답변2
답변3
답변4
범위(분포)

내가 최근에 먹은 것은 김밥, 비빔밥,
냉면, 우동 등이야. 오늘 점심엔
맵지 않은 면 종류로 먹고 싶어.
최근 메뉴와 겹치지 않게
새로운 메뉴 추천해줘.

답변1
답변2

범위(분포)

확률 분포 제어하기 : 프롬프트

✓ LLM의 다음 단어 생성

- 내부적으로 각 단어 후보마다 로짓(logits)을 계산한 뒤, softmax 함수를 사용하여 확률로 변환

✓ Temperature

- 확률 분포 자체를 **날카롭게(0)** 혹은 **넓게(1+)** 조절
 - 낮은 값 (0) : 항상 가장 확률 높은 단어를 고름 → 일관되고 예측 가능한 결과
 - 높은 값 (1) : 낮은 확률 단어도 고를 가능성 있음 → 창의적이지만 불안정한 결과
- 일반적인 값의 범위

값	의미	사용 예시
0	가장 보수적, 항상 같은 답	코드 생성, 논리적 답변
0.3 ~ 0.7	적당한 창의성	일상 대화, 설명문
1 ~	매우 창의성	시, 이야기, 브레인스토밍

$$P(x_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



Temperature > 0

$$P_T(x_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

Temperature = 1

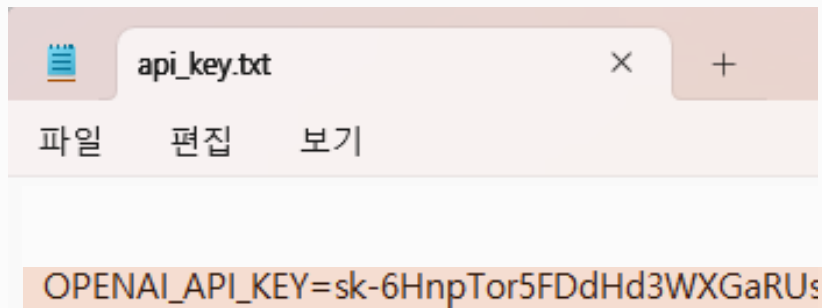
argmax(probabilities)

| LLM API로 사용하기

API 키 등록

✓ API Key를 환경변수에 추가(api_key.txt)

- OPENAI_API_KEY=sk-6Hnp...



```
def load_api_keys(filepath="api_key.txt"):
    with open(filepath, "r") as f:
        for line in f:
            line = line.strip()
            if line and "=" in line:
                key, value = line.split("=", 1)
                os.environ[key.strip()] = value.strip()

# API 키 로드 및 환경변수 설정
load_api_keys('api_key.txt')
```


Model 사용하기

✓ OpenAI API

- OpenAI에서 제공하는 모델 사용
 - gpt-4.1-mini (가성바값!)

```
from langchain_openai import ChatOpenAI
chat = ChatOpenAI(model_name = 'gpt-4.1-mini')
```

- 모델 사용
 - 실행 시 **.invoke**(메시지) 사용

```
result = chat.invoke("세계에서 가장 큰 산은?")
result.content
```

세계에서 가장 큰 산은 에베레스트 산(Mount Everest)입니다. 에베레스트 산의 해발 고도는 약 8,848.86 미터로, 지구상에서 가장 높은 산으로 알려져 있습니다. 이 산은 네팔과 중국(티베트 자치구)의 경계에 위치해 있습니다.

Model 사용하기

✓ 결과 열어보기

result

```
AIMessage(  
  content='세계에서 가장 큰 산은 에베레스트 산(Mount Everest)입니다. 에베레스트 산의 해발 고도는 약 8,848.86미터로,  
  지구상에서 가장 높은 산으로 알려져 있습니다. 이 산은 네팔과 중국(티베트 자치구)의 경계에 위치해 있습니다.',  
  additional_kwargs={'refusal': None},  
  response_metadata={'token_usage': {'completion_tokens': 73, 'prompt_tokens': 14, 'total_tokens': 87,  
    'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0,  
      'reasoning_tokens': 0, 'rejected_prediction_tokens': 0},  
    'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}},  
    'model_provider': 'openai', 'model_name': 'gpt-4.1-mini-2025-04-14',  
    'system_fingerprint': 'fp_d45f83c5fd', 'id': 'chatcmpl-DbR6Rzt09Sj3Zuf1FplxoEFiuUU50',  
    'service_tier': 'default', 'finish_reason': 'stop', 'logprobs': None},  
  id='lc_run--019dee03-5ae6-7453-b8e3-b368e2f3842d-0', tool_calls=[], invalid_tool_calls=[],  
  usage_metadata={'input_tokens': 14, 'output_tokens': 73, 'total_tokens': 87,  
    'input_token_details': {'audio': 0, 'cache_read': 0},  
    'output_token_details': {'audio': 0, 'reasoning': 0}}  
)
```

메시지 구성 방법

✓ 메시지 종류

- SystemMessage : AI에게 주는 지침
- HumanMessage : 사용자 질문 또는 요청
- AIMessage : AI 응답

✓ 메시지 구성

- ① 가장 간단한 입력

```
result = chat.invoke("세계에서 가장 큰 산은?")
```

- ② 일반적인 방식 : 리스트 안에 HumanMessage, SystemMessage로 구성

```
[HumanMessage(content = "세계에서 가장 큰 산은?")]
```

- ③ 멀티모달 입력인 경우

```
[HumanMessage(content = [ {"type": "text", "text": "세계에서 가장 큰 산은?"} ] )]
```

메시지 구성 방법

✓ 메시지 입력 방식

- 튜플 방식

```
s_msg = "너는 애국심을 가지고 있는 건전한 대한민국 국민이야."  
h_msg = "독도는 어느나라 땅이야?"  
  
result = chat.invoke([("system", s_msg), ("human", h_msg)])
```

- 객체 방식 (SystemMessage)

```
result = chat.invoke([SystemMessage(s_msg), HumanMessage(h_msg)])
```

프롬프트 엔지니어링

✓ 프롬프트 엔지니어링이란?

- GPT에게 정확하고 유용한 응답을 얻기 위해 질문(프롬프트)을 설계하는 기술

✓ 왜 중요할까?

- GPT는 모호한 질문에 모호하게 답함
- 정확한 출력은 좋은 입력(Prompt)에서 시작됨
- 역할 부여 / 조건 명시 / 맥락 제공 등으로 출력 퀄리티 차이 발생

✓ GPT의 작동 방식 간단 이해

- GPT는 가장 그럴듯한 다음 단어를 예측
- 따라서 **명확한 방향 제시**가 중요함
 - → 예: "보고서 써줘" vs "3단 구성의 요약 보고서를 작성해줘 (500자 이내)"

생성형 AI의 답변 확률분포에 영향을 줄 수 있는 방법

✓ 프롬프트 엔지니어링

▪ 프롬프트 기본 구조

구성요소	설명 예시
(1) 역할 부여	“넌 제조 분야 데이터 분석 전문가야”
(2) 내용 구성 맥락(Context) 및 목적(Goal) 포함	“2024년 영업 데이터를 바탕으로... 간결한 요약 보고서를 작성해줘”
(3) 답변 형식 지정	“항목별 bullet-point로 정리해줘”

프롬프트 엔지니어링

✓ 마크다운 문법 권장

- #, ##, ###, ... : 제목
- -, *, 1, 2, 3 : 블릿포인트 형식
- **... ** : 강조(굵게 표시)

✓ 예시

역할

너는 데이터 분석 전문가다.

요청사항

- 2026년 수요 데이터를 기반으로 주요 인사이트를 도출해
- 의사결정을 위한 간결한 요약 보고서 작성해

출력형식

- 핵심 인사이트를 항목별 bullet point로 정리해
- 각 항목은 1~2문장으로 간결하게 작성해

프롬프트 엔지니어링

✓ System Message

- 역할 부여
- 지침
- 출력 형식

✓ Human Message

- 요청 사항

```
sys_msg = '''
# 역할
너는 입력하는 도시와 여행 기간을 입력 받아 여행 계획을 수립하는 여행 플래너야

# 지침
- 도시에서 유명한 관광지를 먼저 찾아
- 여행기간을 감안하여 일정을 수립해

# 출력 형식
- bullet point 형식
- 일정별 오전, 오후로 나눠서 간결하게 설명
'''

human_msg = '''
# 여행 도시 : 부산
# 여행 기간 : 2박3일
'''

result = chat.invoke([SystemMessage(sys_msg), HumanMessage(human_msg)])
print(result.content)
```

- 1일차
 - 오전: 해운대 해수욕장 산책 및 해운대 달맞이길 탐방
 - 오후: 동백섬 방문, 아쿠아리움 관람
- 2일차
 - 오전: 감천문화마을 탐방 및 벽화 거리 산책
 - 오후: 자갈치시장 방문, 국제시장 쇼핑 및 먹거리 체험
- 3일차
 - 오전: 태종대 공원 트레킹 및 전망대 방문
 - 오후: 광안리 해변 산책 및 광안대교 야경 감상

LLM의 출력 형식 다루기

✓ LLM 호출 후 출력의 Type

- LLM의 답변은 텍스트(string)

```
print(type(result), type(result.content))
```

```
<class 'langchain_core.messages.ai.AIMessage'> <class 'str'>
```

- 하지만 프로그램 안에서 답변을 사용하려면,
 - 문자열 처리 뿐만 아니라
 - 데이터로 다룰 수 있어야 함(리스트, 딕셔너리, JSON 등)

LLM의 출력 형식 다루기

✓ 문자열을 파이썬 객체로 변환 : `ast.literal_eval()`

- 문자열 형태의 Python dict/list를 실제 객체로 변환

```
import ast
dict2 = ast.literal_eval(result.content)
```

- 특징
 - 구현 간단
 - 빠른 프로토타이핑에 적합
- 한계
 - 출력이 조금만 깨져도 파싱 실패
 - 구조 검증 불가 (필드 누락/타입 오류 대응 어려움)

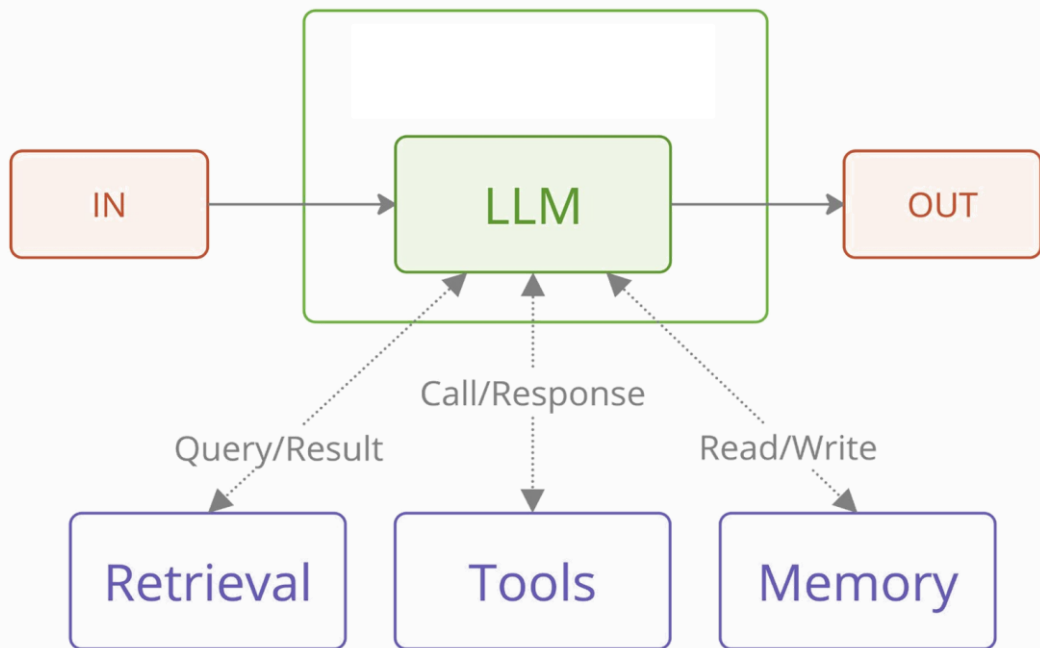
AI Agent 기초

| AI Agent 기본 구조

AI Agent란?

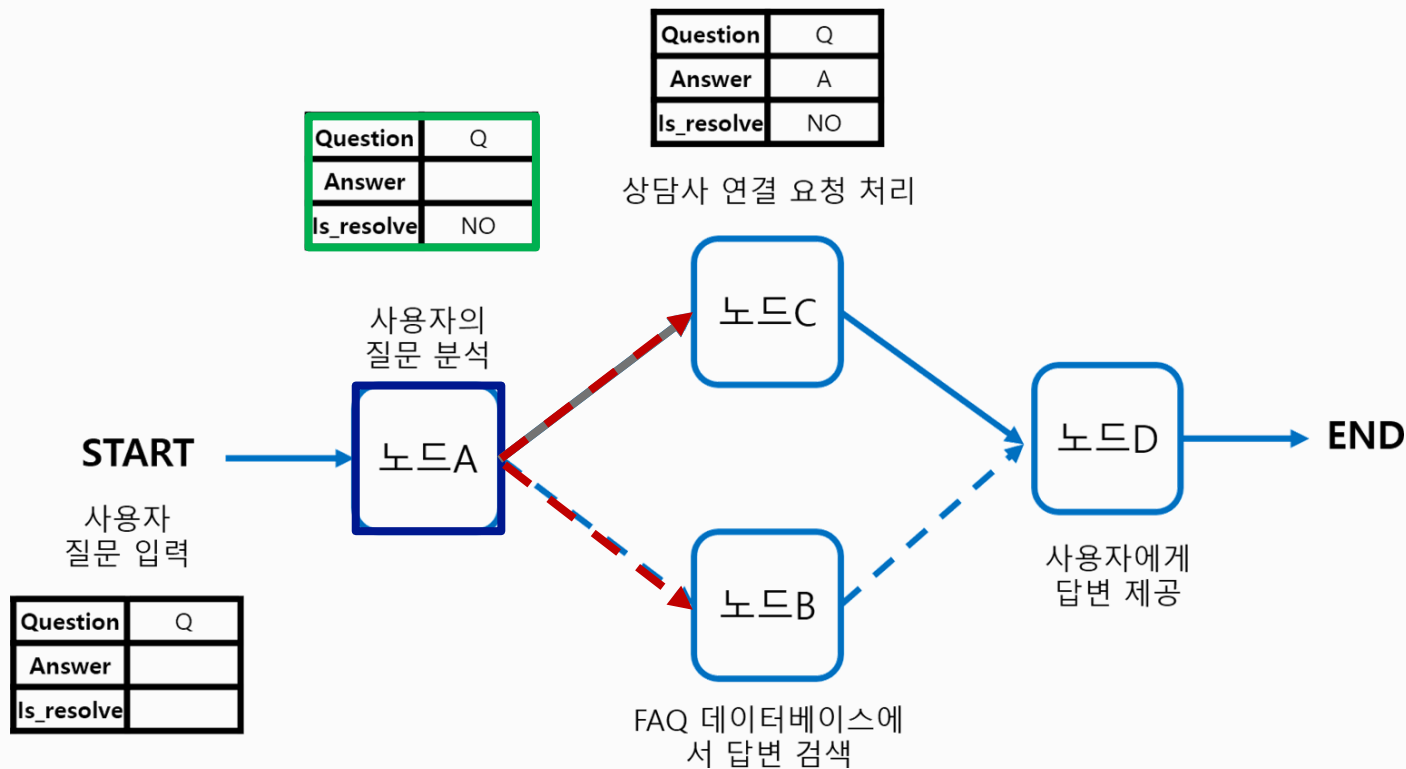
✓ LLM은 사용자의 입력을 받아 응답을 생성

- 필요 시,
 - 외부에서 정보를 검색하거나(Retrieval)
 - 툴을 사용하거나(Tool)
 - 메모리를 읽고 쓰며(Memory)
 - 결과를 종합해서 응답을 생성



Agent 주요 구성 요소

✓ Node, Edge, State, Conditional Edge



AI Agent 구축 절차

- ✓ ① State 정의 및 LLM 준비
- ✓ ② 노드 준비
- ✓ ③ 그래프 구성

① State 정의 및 LLM 준비

✓ State

- 노드의 입력과 출력 관리하는 **딕셔너리 형태**의 자료형
- 그래프를 통과하며 **컨텍스트**를 유지

▪ MessageState

- LangGraph가 기본 제공하는 **State** 템플릿
- llm과 주고 받는 **message** 들을 누적 관리하는 key 포함

```
from langgraph.graph import MessagesState

class SimpleState(MessagesState):
    response: str
```

State 구조

Key	Type	Value
messages	list	
response	str	

② 노드 준비

✓ Node 입, 출력

- 입력 : State (: SimpleState ➡ Type 힌트)
- 출력(return) : State 업데이트
 - 업데이트 할 key 만 지정 ➡ 노드 실행 결과로 업데이트 된 State가 다음 노드로 전달
 - "messages": result ➡ 기존 message 리스트에 result (AI Message) 추가

```
def chatbot(state: SimpleState):  
    # 1. State에서 필요한 정보 저장  
    prompt = state["messages"]  
  
    # 2. prompt 구성  
  
    # 3. llm 호출  
    result = llm.invoke(prompt)  
  
    # 4. state 업데이트  
    return {"messages": result, "response": result.content}
```

State 구조

Key	Type	Value
messages	list	
response	str	

② 노드 준비

✓ Node 내부 설계

- 1. State에서 필요한 정보 뽑아서 저장하기
- 2. prompt 구성
- 3. llm 호출
- 4. state 업데이트

```
def chatbot(state: SimpleState):  
    # 1. State에서 필요한 정보 저장  
    prompt = state["messages"]  
  
    # 2. prompt 구성  
  
    # 3. llm 호출  
    result = llm.invoke(prompt)  
  
    # 4. state 업데이트  
    return {"messages": result, "response": result.content}
```

State 구조

Key	Type	Value
messages	list	
response	str	

③ 그래프 구성

- 그래프 초기화
- Node 추가: 생성한 함수를 노드로 추가. (**add_node**)
- 노드 연결 (**.add_edge**)
- 컴파일

```
# 그래프 초기화
builder = StateGraph(SimpleState)

# 노드 추가
builder.add_node("chatbot", chatbot)

# 엣지 추가
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)

# 컴파일
graph = builder.compile()
```

State 구조

Key	Type	Value
messages	list	
response	str	

그래프 실행

✓ 초기 state 준비

- 필요한 키만 딕셔너리 형태로 지정

✓ 그래프 실행

- graph.invoke()

```
# 초기 state 준비
initial_state = {"messages": "오늘 서울 기온은?"}

# 그래프 실행
result = graph.invoke(initial_state)
```

State 구조

Key	Type	Value
messages	list	
response	str	

그래프 실행

✓ 실행 결과

```
result['response']
```

죄송하지만, 저는 실시간 날씨 정보에 접근할 수 없습니다. 서울의 현재 기온을 확인하시려면:
- **기상청 웹사이트** (www.kma.go.kr) - **날씨 앱** (네이버 날씨, 카카오 날씨 등) - **포털 사이트** (구글, 네이버, 다음 등에서 "날씨" 검색) 이런 방법들을 이용하시면 정확한 현재 기온과 날씨 정보를 바로 확인할 수 있습니다. 혹시 날씨와 관련된 다른 정보가 필요하신가요?

```
for m in result["messages"]:  
    m.pretty_print()
```

===== Human Message =====

오늘 서울 기온은?

===== Ai Message =====

죄송하지만, 저는 실시간 날씨 정보에 접근할 수 없습니다. 서울의 현재 기온을 확인하시려면:
- **기상청 웹사이트** (www.kma.go.kr) - **날씨 앱** (네이버 날씨, 카카오 날씨 등) - **포털 사이트** (구글, 네이버, 다음 등에서 "날씨" 검색) 이런 방법들을 이용하시면 정확한 현재 기온과 날씨 정보를 바로 확인할 수 있습니다. 혹시 날씨와 관련된 다른 정보가 필요하신가요?

State 구조

Key	Type	Value
messages	list	
response	str	

| State 이해

State

✓ Agent에서의 State

- State는 공유 메모리가 아니라 **합의된 데이터 계약**
- 각 노드의 입력과 출력 : State
 - 입력 : State가 그대로 입력됨
 - 출력 : State에 필요한 키만 업데이트

✓ State 키 업데이트 방식

- 덮어쓰기(Overwrite) : 기존 값을 덮어 씌
- 쌓기(Reducer) : 리스트 형식, 값을 추가해 감



각 서랍의 용도가 결정된 서랍장

State 이해하기 예제

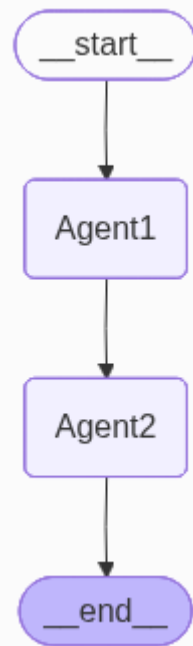
✓ 사용자 입력 내용을 기반으로 다음을 처리하는 Agent를 생성합니다.

- Agent1 : key_point 도출
- Agent2 : 후속조치 도출(2개 리스트 형식)

✓ State 정의

```
class AgentState(MessageState):  
    user_input: str  
    key_point: str  
    action_item: list
```

키	구분	형식
messages	쌓기	리스트
user_input	덮어쓰기	문자열
key_point	덮어쓰기	문자열
action_item	덮어쓰기	리스트

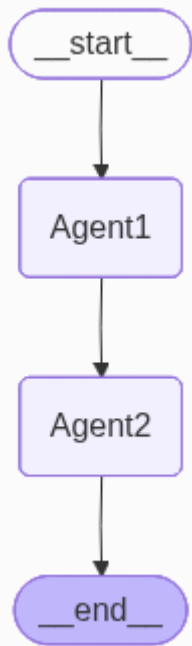


State 이해하기 예제

✓ Node1

```
def keypoint_agent(state: AgentState):  
    # 입력 State 출력  
    print('입력값(초기값)')  
    pprint(result, width=100, sort_dicts=False)  
    print('='*200)  
  
    # 1. State로 부터 필요한 정보 가져오기  
    user_input = state['messages'][-1].content  
  
    # 2. 프롬프트 구성  
    system = SystemMessage(''  
        # 역할 : 사용자 입력에서 핵심 문제점 2개를 도출하는 에이전트.  
        # 출력 : 블릿포인트 형식.\n''')  
    human = HumanMessage(content=user_input)  
  
    # 3. llm 호출  
    result = llm.invoke([system, human])  
  
    # 4. State 업데이트  
    return {"messages": result, "user_input": user_input, "key_point": result.content}
```

키	구분	형식
messages	쌓기	리스트
user_input	덮어쓰기	문자열
key_point	덮어쓰기	문자열
action_item	덮어쓰기	리스트

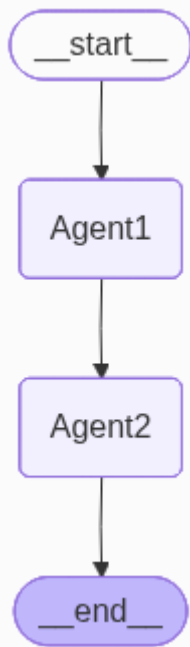


State 이해하기 예제

✓ Node2

```
def solution_agent(state: AgentState):  
    # 입력 State 출력  
    print('Agent1 실행 결과')  
    pprint(result, width=100, sort_dicts=False)  
  
    # 1. State로 부터 필요한 정보 가져오기  
    user_input = state.get("user_input")  
    key_point = state.get("key_point")  
  
    # 2. 프롬프트 구성  
    system = SystemMessage('''  
        # 역할 : 핵심 문제점에 대한 후속 조치를 2개를 도출하는 에이전트.  
        # 출력형식  
        - 답변에 후속조치를 담는 리스트(list) 외에 어떠한 답변도 달지마.  
        - 예 : ['후속조치1', '후속조치2']''')  
    human = HumanMessage(f'''  
        * 사용자 입력:\n{user_input}  
        * 핵심 문제점:\n{key_point}''')  
  
    # 3. llm 호출  
    result = llm.invoke([system, human])  
    action = ast.literal_eval(result.content)  
  
    # 4. State 업데이트  
    return {"messages": result, "action_item": action}
```

키	구분	형식
messages	쌓기	리스트
user_input	덮어쓰기	문자열
key_point	덮어쓰기	문자열
action_item	덮어쓰기	리스트



State 이해하기 예제

✓ 실행

- 각 노드에 print문(pprint 문)으로 중간 과정 및 최종 결과 출력하기

```
text = '''이번 달 말에 대대적으로 출시하기로 한 신규 모바일 앱 프로젝트가 정말 큰 위기에 봉
착했습니다.
```

```
어제 밤사이 메인 개발 서버가 갑자기 최신형 랜섬웨어 공격을 받아서 모든 소스 코드와 DB가 암호
화되어 버렸습니다.
```

```
보안 팀에서 급하게 복구를 시도해봤지만, 해커가 요구하는 암호화 해제 비용이 예산을 초과하여
사실상 기술적 복구가 불가능하다는 결론을 내렸습니다.'''
```

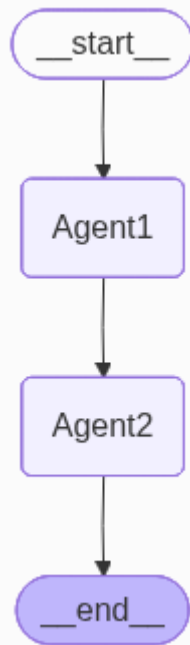
```
# 초기 State 설정 및 실행
```

```
initial_state : AgentState = {"messages": [HumanMessage(text)]}
```

```
result = graph.invoke(initial_state)
```

```
# 최종 결과 출력
```

```
pprint(result, width=100, sort_dicts=False)
```



State 이해하기 예제

중간
State

최종
State

입력값(초기값)
{'messages': [HumanMessage(content='이번 달 말에 대대적으로 출시하기로 한 신규 모바일 앱 프로젝트가 정말 큰 위기에 봉착했')]

Agent1 실행 결과

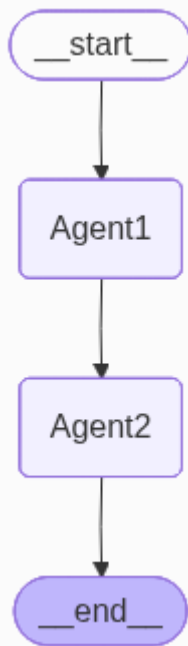
```
{'messages': [HumanMessage(content='이번 달 말에 대대적으로 출시하기로 한 신규 모바일 앱 프로젝트가 정말 큰 위기에 봉착했'),
  AIMessage(content='# 신규 모바일 앱 프로젝트 위기 - 핵심 문제점 2개\n\n* **즉각적 데이터 손실 위기**\n\n- 모
  'user_input': '이번 달 말에 대대적으로 출시하기로 한 신규 모바일 앱 프로젝트가 정말 큰 위기에 봉착했습니다.\n'
  '어제 밤사이 메인 개발 서버가 갑자기 최신형 랜섬웨어 공격을 받아서 모든 소스 코드와 DB가 암호화되어 버렸습니다.\n'
  '보안 팀에서 급하게 복구를 시도해봤지만, 해커가 요구하는 암호화 해제 비용이 예산을 초과하여 사실상 기술적
  '내렸습니다.\n'),
  'key_point': '# 신규 모바일 앱 프로젝트 위기 - 핵심 문제점 2개\n'
  '\n'
  '* **즉각적 데이터 손실 위기**\n'
  '  - 모든 소스 코드와 DB 암호화로 인한 완전 손실 상태\n'
  '  - 기술적 복구 불가능(비용 초과로 인한 암호 해제 불가)\n'
  '  - 월말 출시 데드라인까지 복구 불가능성 높음\n'
  '\n'
  '* **사업 연속성 및 일정 위기**\n'
  '  - 대대적 출시 예정으로 이미 진행된 마케팅/리소스 배분\n'
  '  - 소스 코드 전체 재구성 필요 시 현실적 개발 일정 재검토 필수\n'
  '  - 출시 연기에 따른 사업상 손실, 이해관계자 신뢰 훼손 가능성'}
```

```
{'messages': [HumanMessage(content='이번 달 말에 대대적으로 출시하기로 한 신규 모바일 앱 프로젝트가 정말 큰 위기에 봉착했습니다'),
  AIMessage(content='# 신규 모바일 앱 프로젝트 위기 - 핵심 문제점 2개\n\n* **즉각적 데이터 손실 위기**\n\n- 모든 소
  AIMessage(content='[긴급 백업 시스템 재검토 및 복구 가능한 대체 데이터 소스(깃허브, 클라우드 백업, 팀 로컬 환경)
  'user_input': '이번 달 말에 대대적으로 출시하기로 한 신규 모바일 앱 프로젝트가 정말 큰 위기에 봉착했습니다.\n'
  '어제 밤사이 메인 개발 서버가 갑자기 최신형 랜섬웨어 공격을 받아서 모든 소스 코드와 DB가 암호화되어 버렸습니다.\n'
  '보안 팀에서 급하게 복구를 시도해봤지만, 해커가 요구하는 암호화 해제 비용이 예산을 초과하여 사실상 기술적 복구가
  '내렸습니다.\n'),
  'key_point': '# 신규 모바일 앱 프로젝트 위기 - 핵심 문제점 2개\n'
  '\n'
  '* **즉각적 데이터 손실 위기**\n'
  '  - 모든 소스 코드와 DB 암호화로 인한 완전 손실 상태\n'
  '  - 기술적 복구 불가능(비용 초과로 인한 암호 해제 불가)\n'
  '  - 월말 출시 데드라인까지 복구 불가능성 높음\n'
  '\n'
  '* **사업 연속성 및 일정 위기**\n'
  '  - 대대적 출시 예정으로 이미 진행된 마케팅/리소스 배분\n'
  '  - 소스 코드 전체 재구성 필요 시 현실적 개발 일정 재검토 필수\n'
  '  - 출시 연기에 따른 사업상 손실, 이해관계자 신뢰 훼손 가능성',
  'action_item': '[긴급 백업 시스템 재검토 및 복구 가능한 대체 데이터 소스(깃허브, 클라우드 백업, 팀 로컬 환경) 즉시 확인',
  '월말 출시 일정 공식 재협의 및 단계적 출시 계획(MVP 기반 축소 출시-후후 업데이트) 수립을 통한 이해관계자 커뮤니케이션 실시']}]
```

키	구분
messages	쌓기
user_input	덮어쓰기
key_point	덮어쓰기
action_item	덮어쓰기

키	구분
messages	쌓기
user_input	덮어쓰기
key_point	덮어쓰기
action_item	덮어쓰기

키	구분
messages	쌓기
user_input	덮어쓰기
key_point	덮어쓰기
action_item	덮어쓰기



요약 | Overview

✓ LLM : 확률분포 기반 답변 생성 모델

- 답변이 비 결정적임 ➡ 답변에 영향을 주는 방법 : 프롬프트

✓ Agent 구축 절차

- ① State 정의 및 LLM 준비
- ② 노드 준비 : State로 부터 필요 정보 추출 / 메시지 구성 / llm 호출 / 결과를 이용하여 State 업데이트
- ③ 그래프 구성

✓ Message 종류

- System Message, Human Message, AI Message, Tool Message

✓ 출력 다루기

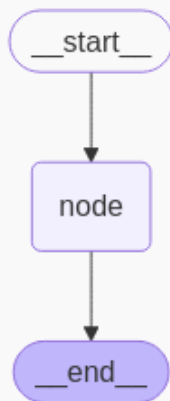
- LLM의 답변은 문자열
- 문자열이 아닌 파이썬 자료형으로 사용하려면,
 - 프롬프트에서 출력 양식 지정(예시 포함)
 - `ast.literal_eval()`로 변환

AI Agent 기본 패턴

기본 패턴

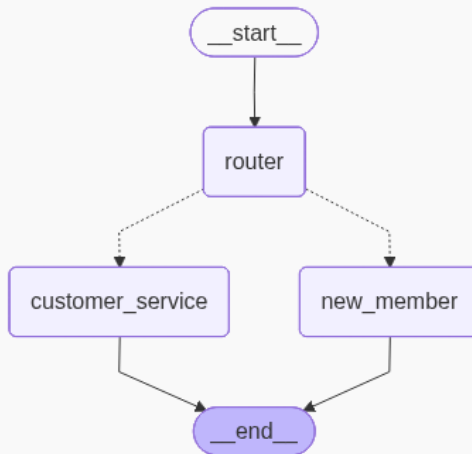
Graph① 순차 흐름

- 입력, 처리, 출력으로 이어지는 단순 흐름



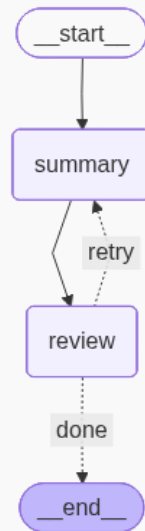
Graph② 라우팅

- 특정 입력에 따라 서로 다른 경로를 선택하여 실행 흐름을 제어
- 사용자의 입력이나 특정 조건에 따라 서로 다른 노드 실행



Graph③ 반복루프

- 조건에 따라 반복을 수행하는 구조
- 에이전트는 반복 수행 속에서 점진적으로 성능을 향상



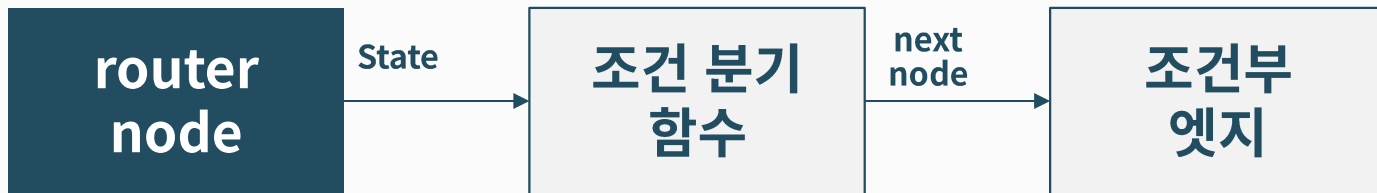
| Routing 패턴

Routing 패턴 중점 사항

✓ 의사결정 노드 : router

- 다음 실행 노드 결정 → 결과는 State에 저장
 - 규칙 기반 결정
 - llm이 결정

✓ 조건 분기 절차



```
builder.add_conditional_edges("router", condition, {  
    "new_member": "new_member", "customer_service": "customer_service"})
```

노드 이름

조건 분기 함수 결과

State 정의

✓ 그래프 구성

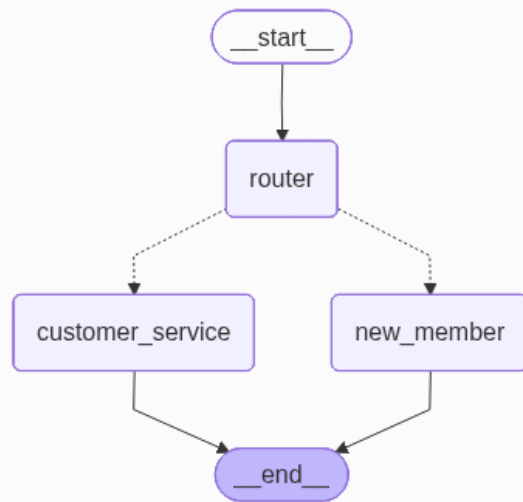
- 고객 요청사항을 접수 받아
 - 신규 가입이면 ➡ new_member agent로
 - 그 외 문의사항 ➡ customer_service agent
- 각 agent 처리 후 종료

✓ State

```
class SimpleState(MessagesState):  
    next_agent: str  
    response: str
```

State

Key	Type	Value
messages	list	
next_agent	str	
response	str	



Node 준비

```
def CS_agent(state: SimpleState):  
    # 1. State에서 필요한 정보 저장  
    prompt = state["messages"][0].content
```

메시지 리스트에서
첫번째 항목[0]의 내용(content)

```
    # 2. prompt 구성  
    sys_msg = SystemMessage(''  
    # 역할  
    너는 고객 요청사항 접수 담당 에이전트야.
```

```
    # 지침  
    - 고객 요청사항을 받아서 담당자를 지정해.
```

```
    # 출력 형식  
    - new_member 혹은 customer_service 둘 중 하나만 출력해.  
    '')  
    hu_msg = HumanMessage(prompt)
```

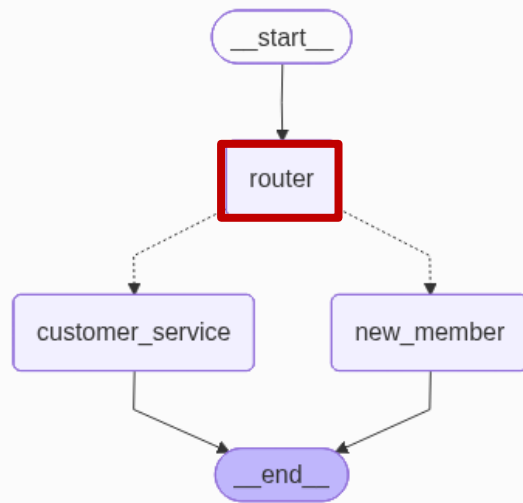
```
    # 3. llm 호출  
    result = llm.invoke([sys_msg, hu_msg])
```

```
    # 4. state 업데이트  
    return {"messages": result, "next_agent": result.content}
```

messages에 AI Message 추가 next_agent에 AI Message의 내용 업데이트

State

Key	Type	Value
messages	list	
next_agent	str	
response	str	



Node 준비

```
# 신규 가입 agent
def new_member(state: SimpleState):
    # 1. State에서 필요한 정보 저장

    # 2. prompt 구성

    # 3. llm 호출
    result = AIMessage('요청하신 신규 가입 건이 처리 되었습니다.')

    # 4. state 업데이트
    return {"messages": result, "response": result.content}

# 일반 지원 agent
def customer_service(state: SimpleState):
    # 1. State에서 필요한 정보 저장

    # 2. prompt 구성

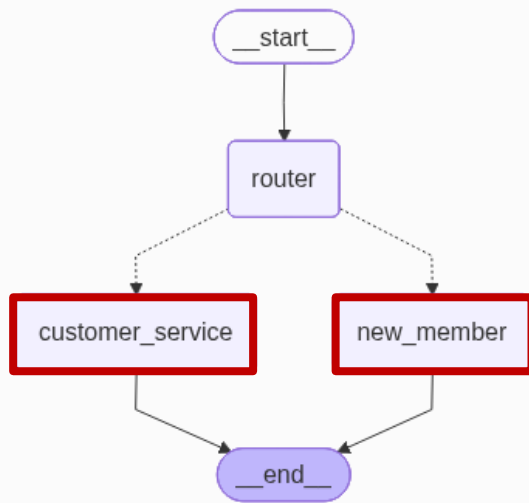
    # 3. llm 호출
    result = AIMessage('요청하신 요청 건이 처리 되었습니다.')

    # 4. state 업데이트
    return {"messages": result, "response": result.content}
```

여기에서는 두 Agent가
고정된 답변을 하도록 간단히 구현합니다.

State

Key	Type	Value
messages	list	
next_agent	str	
response	str	



그래프

그래프 생성

```
builder = StateGraph(SimpleState)
```

노드 등록

```
builder.add_node("router", CS_agent)
```

```
builder.add_node("new_member", new_member)
```

```
builder.add_node("customer_service", customer_service)
```

```
def condition(state: SimpleState):
```

```
    return state['next_agent']
```

조건 분기 함수

조건 분기 연결

```
builder.add_conditional_edges("router", condition,  
    {"new_member": "new_member", "customer_service": "customer_service"})
```

키와 값이 같다면 생략 가능

나머지 연결

```
builder.add_edge(START, "router")
```

```
builder.add_edge("new_member", END)
```

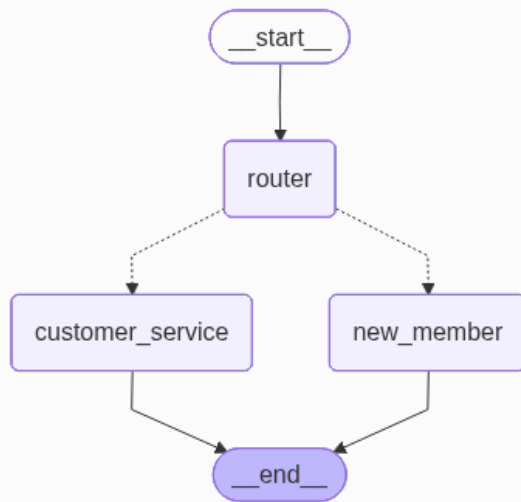
```
builder.add_edge("customer_service", END)
```

그래프 컴파일

```
graph = builder.compile()
```

State

Key	Type	Value
messages	list	
next_agent	str	
response	str	



| Loop 패턴

Loop 패턴 중점 사항

✓ 반복 여부 결정

- Router 패턴과 유사

✓ 무한 반복 방지 장치

- 반복 횟수 제한

State 정의

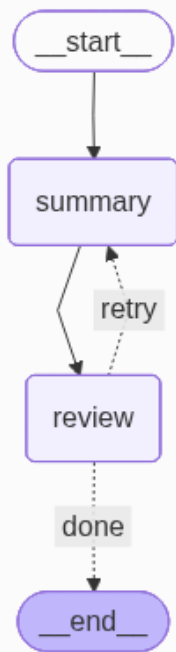
✓ 그래프 구성

- 예시 : 문서(긴 문장) 입력 > 요약 > 검토
 - 적합 : 종료(done)
 - 부적합 : 다시 요약(retry)

✓ State

- Loop에서는 언제 반복을 종료할지 기준이 중요
 - 기본 방법 : 반복횟수 제한하기

```
class SummaryState(MessagesState):  
    input_text: str          # 원본 입력  
    summary: str             # 요약된 문장  
    satisfied: str           # 요약 결과 평가 : 적합, 부적합  
    loop_count: int          # 반복횟수
```



Graph③ Loop : Node 준비

```
def summary_agent(state: SummaryState):
```

```
# 1. State에서 필요한 정보 저장
```

```
prompt = state["input_text"]
```

```
curr_count = state["loop_count"]
```

만약, loop_count에 값이 없다면, 초기값 0 부여
state.get('loop_count', 0)

```
# 2. prompt 구성
```

```
sys_msg = SystemMessage(''
```

```
# 역할
```

```
너는 입력받은 내용을 요약하는 에이전트야.
```

```
# 지침
```

```
- 간결하게 3가지 항목으로 요약해.
```

```
# 출력 형식
```

```
- Bullet-point
```

```
''')
```

```
hu_msg = HumanMessage(prompt)
```

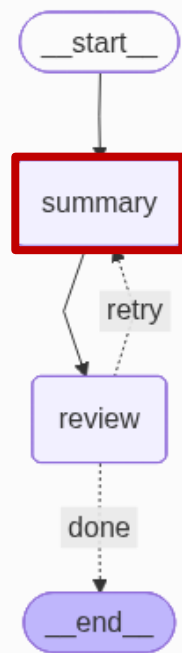
```
# 3. llm 호출
```

```
result = llm_summary.invoke([sys_msg, hu_msg])
```

```
# 4. state 업데이트
```

```
return {"messages": result, "summary": result.content, 'loop_count': curr_count+1}
```

반복횟수 업데이트



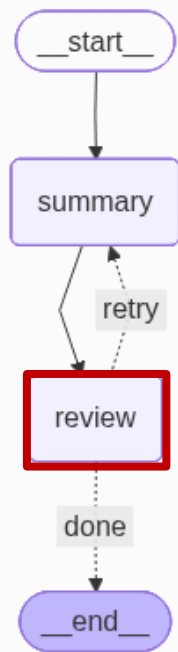
Graph③ Loop : Node 준비

```
def review_agent(state: SummaryState):  
    # 1. State에서 필요한 정보 저장  
  
    # 2. prompt 구성  
  
    # 3. llm 호출  
    result = AIMessage('부적합')  
  
    # 4. state 업데이트  
    return {"messages": result, "satisfied": result.content}
```

여기에서는 단순히 부적합 판정하도록 구성

```
def condition(state: SummaryState):  
    satisfied = state['satisfied']  
    curr_count = state["loop_count"]  
  
    # 적합 이거나, 재시도 횟수가 2을 초과하면 done  
    if (satisfied == '적합') or (curr_count > 2):  
        next_step = 'done'  
    else :  
        next_step = 'retry'  
  
    return next_step
```

반복 종료 조건



Graph③ Loop : 그래프

✓ 조건 분기 함수 condition에 따른 반복 제어 그래프

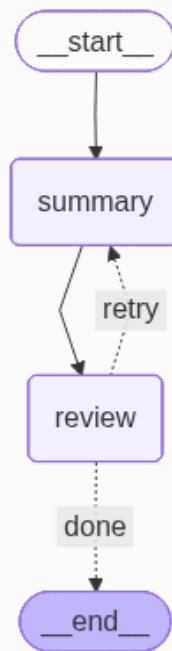
```
# 초기화
builder = StateGraph(SummaryState)

# 2.노드추가
builder.add_node("summary", summary_agent)
builder.add_node("review", review_agent)

# 3.연결
builder.add_edge(START, "summary")
builder.add_edge("summary", "review")

builder.add_conditional_edges("review", condition, {"done": END, "retry": "summary"})

# 4. 컴파일
graph = builder.compile()
```



종합 실습 : Router + Loop 구현하기

✓ 업무 흐름

- 업무 이메일을 수신하면 문의 유형을 분류하고
- 적절히 답변을 생성

[입력] 이메일 내용



[router] 이메일 유형 분류

└─ "문의" → inquiry (문의 답변 초안 작성)

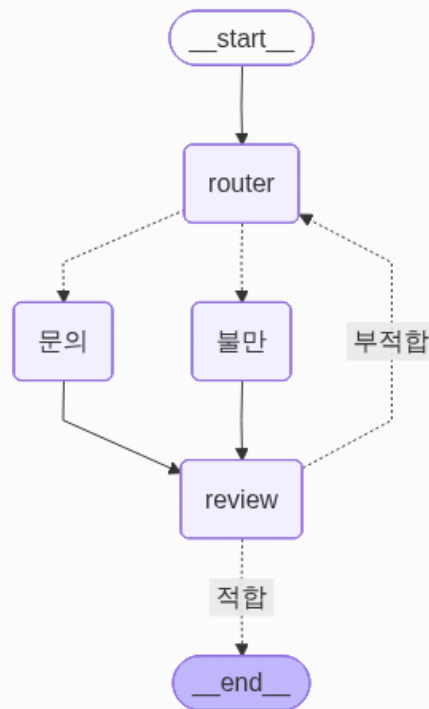
└─ "불만" → complaint (불만 처리 초안 작성)



[review] 초안 품질 검토 (LLM 판단)

└─ "적합" → END

└─ "부적합" → 다시 초안 작성 (최대 3회)



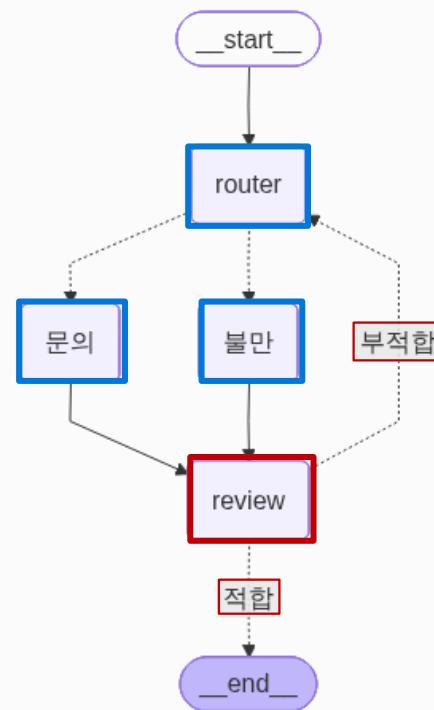
종합 실습 : Router + Loop 구현하기

✓ 2단계로 구현

	목표	그래프 구조	핵심 포인트
1단계	Routing만 동작 확인	START → router → 문의/불만 → END	조건 분기 함수 1개, email_type 흐름 이해
2단계	Loop 추가해서 전체 완성	1단계 + review → 적합/부적합 분기	조건 분기 함수 2개, loop_count 관리, 부적합 → router 이유 이해

✓ State

필드	router	문의 / 불만	review
messages	R/W	R/W	R/W
email_type	W	R	-
draft	-	W	R
loop_count	-	-	R/W



요약 | Overview

✓ Routing 패턴

- 조건에 따라 여러 노드로 경로 지정
- 조건 분기 함수, Conditional Edge

✓ Loop 패턴

- 조건에 따라 반복 수행
- 조건 분기 함수, Conditional Edge
- 반복 횟수 제어

AI Agent 도구와 메모리

| 도구 사용하기

Tools

✓ Tool

- Agent가 특정 작업을 수행하기 위해 호출할 수 있는 외부 기능(함수)
[예: 계산기, 웹 검색, DB 조회, 날씨 확인 등]
- LLM은 학습한 데이터만 알고 있음
→ 필요할 때 적절한 Tool 호출

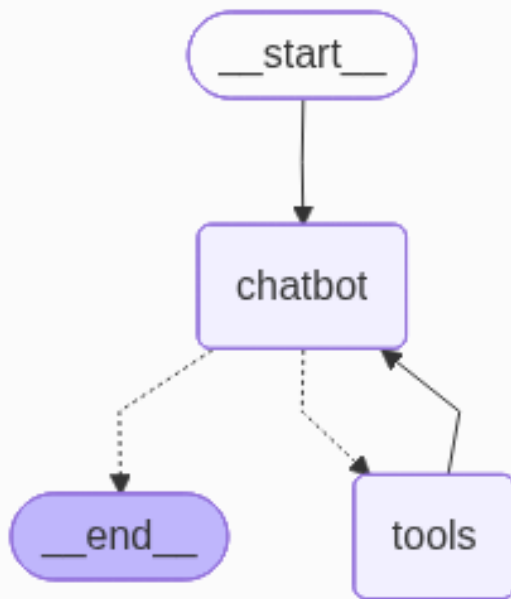
✓ 두가지 유형

- 외부 도구 : 다양한 provider가 제공하는 도구
- Custom Tool : 직접 만든 도구

Tool 사용 Agent

✓ Agent 구성

- 사용자 질문 → chatbot 노드
- chatbot 노드에서 LLM이 판단
 - 도구 불필요 : 직접 답변 생성
 - 도구 필요 : tool_call (도구 호출) → tools 노드 실행
 - 도구 실행결과를 참조하여 llm이 답변 생성

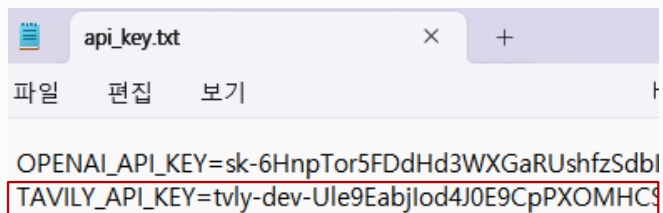


Tool 사용 Agent

✓ Tavily : 구글 검색 도구

▪ API Key 발급 및 등록

- 사이트 접속 및 회원가입 : <https://app.tavily.com/>
- api_key.txt에 추가 : TAVILY_API_KEY=

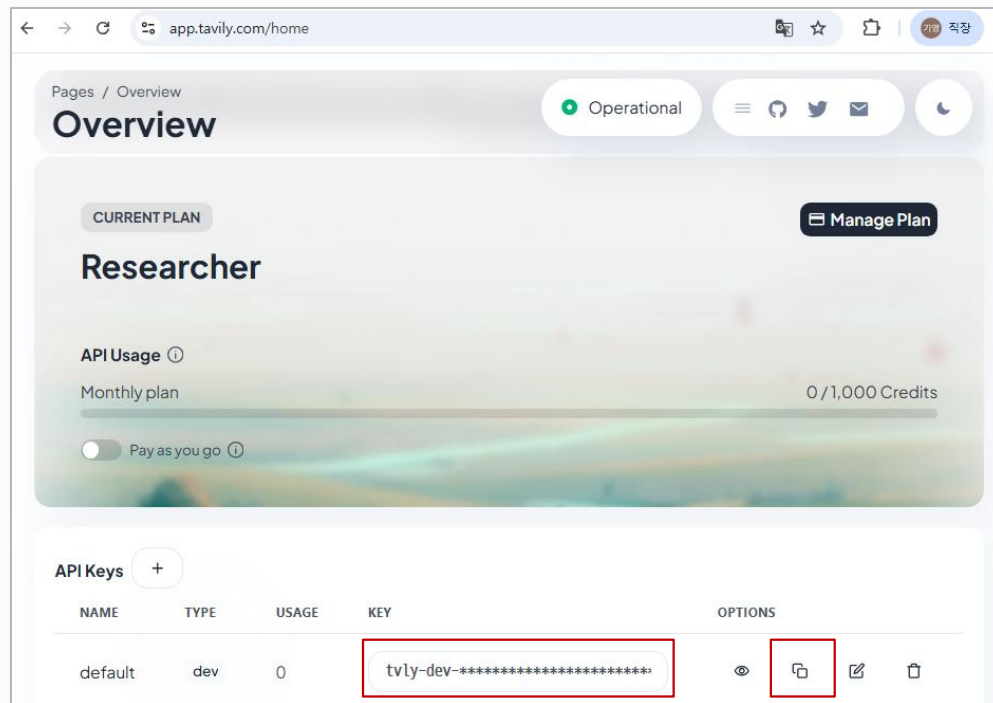


```
api_key.txt
파일 편집 보기

OPENAI_API_KEY=sk-6HnpTor5FDdHd3WXGaRUshfzSdbl
TAVILY_API_KEY=tvly-dev-Ule9Eabjlod4J0E9CpPXOMHC3
```

▪ API 키 로드 및 환경변수 설정

```
load_api_keys(path + 'api_key.txt')
```



복사 버튼

Tool 사용 Agent

✓ 구축 절차

- 도구 사용을 위한 라이브러리 설치

```
!pip install -q langchain-tavily tavily-python
```

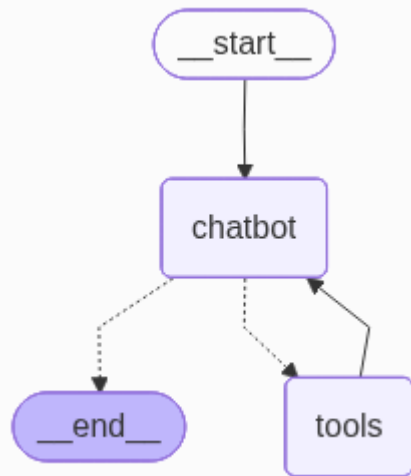
- 도구 준비

```
from langchain_tavily import TavilySearch  
tavily_tool = TavilySearch(max_results=3)
```

- LLM에 도구 붙이기

```
llm = ChatOpenAI(model="gpt-4.1-mini").bind_tools([tavily_tool])
```

- LLM이 도구가 필요하다고 판단 (tool_call)
- 필요 없으면 그냥 답변 생성 (AI Message)



Tool 사용 Agent

✓ 구축 절차

▪ Tool Node 준비

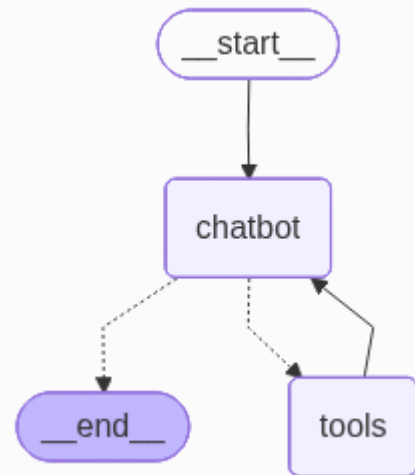
```
from langgraph.prebuilt import ToolNode, tools_condition
tool_node = ToolNode([tavily_tool])
```

- **tools_condition**: langgraph 에서 제공되는 도구 호출 조건 분기 함수
 - llm의 Message가 도구 호출(tool call) 하면 ➡ **'tools'**
 - 아니면 ➡ **'__end__'** (END와 동일한 의미)

```
builder.add_conditional_edges("chatbot", tools_condition,
                             {'tools': 'tools', '__end__': END})
```

생략 가능

- 경우에 따라서, **'__end__'**: '다른 노드' 로 지정 가능

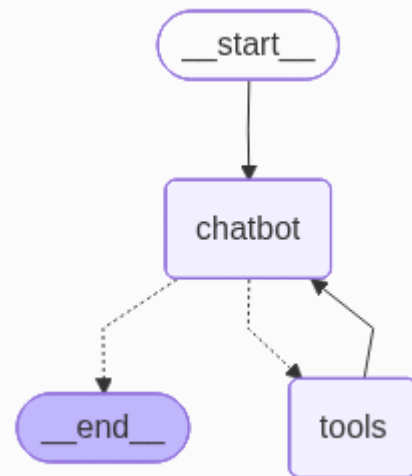


Tool 사용 Agent – 구글 검색 도구(Tavily)

✓ 도구 호출 절차 예시

- Human Message : 요청사항
- AI Message : 검색 도구 사용 결정
 - tool call
 - 도구 호출 시, 해당 도구(함수)의 입력 매개변수에 맞게 **입력값 구성**

```
===== Human Message =====
오늘은 달달한 로맨틱 코미디, 2026년 영화
===== Ai Message =====
[{'text': '2026년의 최신 달달한 로맨틱 코미디 영화를 찾아드리겠습니다!', 'type': 'text'},
 {'id': 'toolu_01CPfJgJqtgJvF9kENL4K42c', 'caller': {'type': 'direct'},
  'input': {'query': '2026년 로맨틱 코미디 영화', 'start_date': '2026-01-01',
   'search_depth': 'advanced'},
  'name': 'tavily_search', 'type': 'tool_use'}]
Tool Calls: tavily_search (toolu_01CPfJgJqtgJvF9kENL4K42c) Call ID:
toolu_01CPfJgJqtgJvF9kENL4K42c Args: query: 2026년 로맨틱 코미디 영화 start_date: 2026-01-
01 search_depth: advanced
===== Tool Message =====
...도구 실행 결과(검색 결과)
```



Tool 사용 Agent – 도구 만들어 사용하기

✓ 도구를 함수로 직접 만들어서 사용 가능

✓ 도구 함수 생성 시

- 함수 선언(def 라인)

- 바로 윗줄에 `@tool` 데코레이터 추가 ➡ 그래프에서 함수로 인식

- 바로 아래로 `docstring` 추가 : 함수에 대한 설명 ➡ LLM이 함수 내용 파악하기 위함

```
@tool
def calculator_tool(expression: str) -> str: # type hint : 함수의 출력 type은 str
    '''수식을 입력 받아 계산하는 도구'''
    result = eval(expression) # Python 내장 함수 eval()을 사용해서 문자열로 된 식을 계산
    return f"계산 결과: {result}"
```

Tool 사용 Agent – 도구 만들어 사용하기

✓ 나머지 절차는 동일

- LLM에 도구 연결 : `.bind_tools()`
- 도구 노드 준비 : `ToolNode`
- 그래프 구성 시 조건분기 함수 : `tools_condition`

종합 실습 : 여행 일정 추천 Agent

✓ 시나리오

- 여행지, 기간, 날짜를 입력하면 최신 정보를 검색해 일정 초안을 작성하고, 품질 검토까지 자동으로 수행하는 Agent

[입력] 여행지 / 기간 / 날짜



[search] Tavily로 핫플레이스 · 맛집 · 숙소 검색



[plan] 검색 결과 기반 날짜별 일정 초안 작성

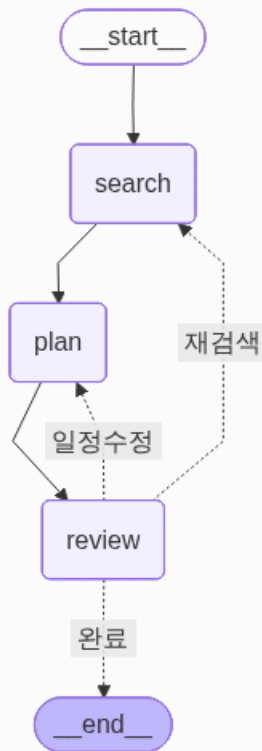


[review] 일정 품질 검토

├─ 완료 → END

├─ 일정수정 → plan (동선 과밀 등 일정 문제)

└─ 재검색 → search (정보 부족)



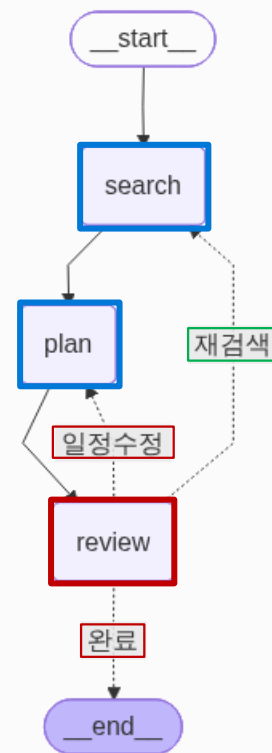
종합 실습 : 여행 일정 추천 Agent

✓ 구현 절차

단계	구현 내용	핵심 사항
1단계	search → plan	Tavily 도구 연동, 순차 흐름
2단계	+ review, 2방향 분기(일정수정, 완료)	Loop 구조, loop_count 관리
3단계	+ 3방향 분기 (재검색 추가)	재검색 → search 재실행

✓ State

필드	search	plan	review
destination / duration / travel_date	R	R	-
search_result	W	R	-
plan	-	W	R
review	-	-	W
loop_count	-	-	R/W

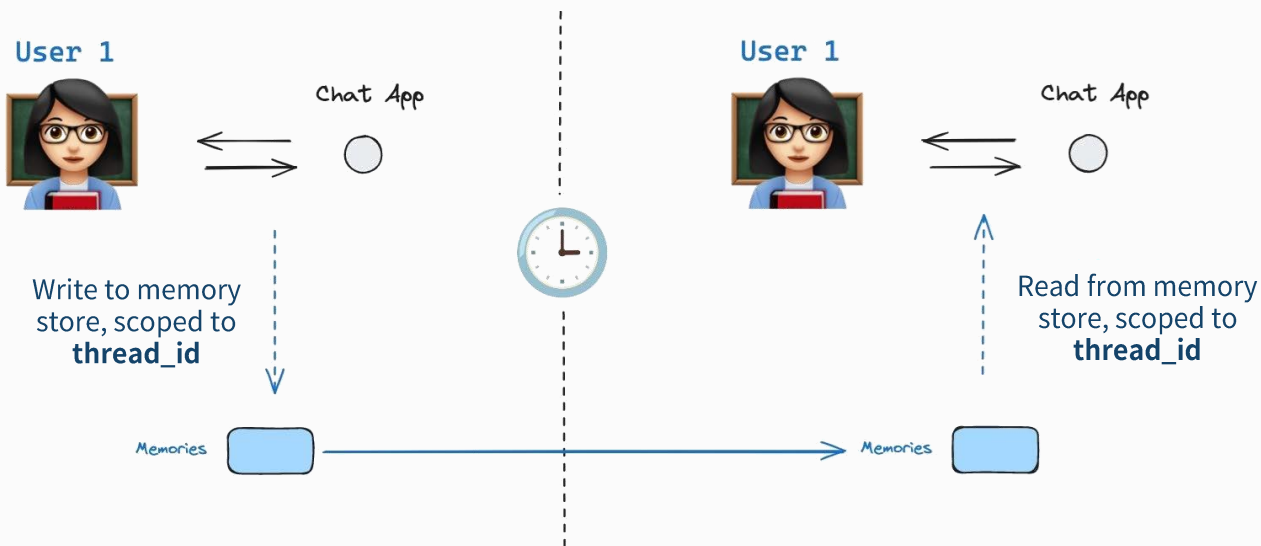


| 메모리 사용하기

Agent에서의 Memory

✓ Memory

- AI Agent가 이전 상태(state)나 대화 기록을 유지하고 재사용할 수 있도록 하는 기능
 - 유저와 AI 간의 대화를 저장하여 **지속적인 맥락을 유지**
 - 단순한 세션 메모리, 외부 저장소를 활용하여 장기적인 상태 저장 가능



단기, 장기 메모리

✓ 단기 메모리 (Checkpoint)

- 실행 중 상태(State)를 시점 단위로 저장하여 재현성과 제어를 보장하는 메커니즘
- 맥락 유지 : 대화 흐름을 Snapshot 형태로 저장
- 상태 복원 (State Recovery) : Snapshot 기반 재실행 가능

✓ 장기 메모리 (Store)

- 정보를 축적하고 활용하는 저장소
- 대화 기반 정보 추출 및 저장
- 교차 세션 활용 (Cross-thread): 검색 기반 활용 (Retrieval), Vector DB 개념
- 지능형 개인화: 과거의 경험을 바탕으로 시간이 지날수록 사용자에게 더 최적화된 맞춤형 응답을 제공하는 에이전트의 지식 베이스 역할

단기 메모리와 실시간 관리

✓ MemorySaver

- LangGraph에서 제공하는 Checkpoint 저장소
- Checkpoint : 그래프의 각 Node가 실행될 때마다 State 전체를 스냅샷으로 저장.
- 주요 특징

용도	▪ 그래프의 State를 저장하고 복원
저장 위치	▪ Python의 메모리(RAM) 안에 저장 (디스크 X)
사용 방식	▪ 그래프 compile 시 checkpointer로 지정
이어서 실행	▪ 이전 실행의 마지막 상태부터 재실행 가능
한계	▪ 세션이 종료되면 저장된 상태는 사라짐

MemorySaver 사용하기

✓ 메모리 세팅 절차

- MemorySaver 선언
- 그래프를 컴파일 할 때, checkpointer 옵션으로 지정

메모리 준비

```
from langgraph.checkpoint.memory import MemorySaver # MemorySaver 모듈 가져오기
memory = MemorySaver() # 메모리 기반 체크포인트 초기화
```

컴파일

```
# 체크포인트와 함께 그래프 컴파일
agent_with_memory = builder.compile(checkpointer = memory)
```

MemorySaver 사용하기

✓ 메모리 사용

- checkpointer = memory 로 컴파일 된 그래프
 - invoke 함수의 두 번째 인자로 thread_id(config 딕셔너리)를 받음
- {"configurable": {"thread_id": "1"}}
 - 체크포인트가 어떤 Thread의 메모리를 불러올지 지정
 - thread_id: 실제 서비스에서는 사용자의 ID나 이메일, 혹은 특정 목적을 담은 문자열.

스레드 지정

```
# 대화 흐름을 구분하기 위한 thread_id 설정
thread_1 = {"configurable": {"thread_id": "1"}}
```

실행

```
messages = [HumanMessage(content="3과 4를 더하라")]
messages = agent_with_memory.invoke({"messages": messages}, thread_1)

messages = [HumanMessage(content="거기에 2를 곱하라.")]
messages = agent_with_memory.invoke({"messages": messages}, thread_1)
```


요약 | Overview

✓ Agent에서의 Tool

- LLM이 외부 도구를 활용해서 답변을 도움
- LLM이 도구를 호출할지 결정, 도구의 결과를 받아서 LLM이 답변 생성

✓ Custom Tool(직접 만들기)

- 함수로 생성시, @tool (데코레이터) 붙이기
- 함수 첫 줄에 docstring 추가(함수 설명문) : LLM은 docstring을 보고 필요한 도구 판단

✓ 외부 Tool

- 각 도구에 맞는 라이브러리 설치 및 로딩, 필요시 API Key 준비

✓ State, Memory

- State : 한번 실행 시, 그래프 내에서 Context 유지
- Memory : 동일한 ID(혹은 thread)로 여러 번 실행 시, 이전 State를 저장·복원하여 대화 맥락을 유지

✓ MemorySaver

- 선언하고 그래프 컴파일 시 checkpointer 로 지정
- 같은 thread id로 대화 시 메모리 내용이 state의 message에 모두 포함 됨